

Control de Posición de Motor DC

Curtale Lautaro Adrián, Torreblanca Agustín

UTN FRA - Técnicas Digitales III

Avellaneda, Argentina

lacurtale@gmail.com

agusalejotorre00@gmail.com

Abstract— El proyecto consiste en el control de posición de un motor de corriente continua mediante la aplicación de un control PID. Tiene implementado un datalogger para guardar resultados y configuraciones y se desarrolló mediante el uso de FreeRTOS en una Raspberry Pi Pico 2 W

II. FUNDAMENTACIÓN

I. ÍNDICE

I. ÍNDICE.....	1
II. FUNDAMENTACIÓN.....	1
A. Problemática.....	1
B. Fundamentación Teórica de Hardware.....	1
C. Fundamentación Teórica de Software.....	2
III. ALCANCE.....	2
IV. DIAGRAMA EN BLOQUES.....	2
V. HARDWARE.....	2
A, Stick Raspberry Pi Pico 2W.....	2
B, Puente H L298N.....	2
C. Display LCD 16x2 I2C.....	3
D, Teclado Matricial 4x4.....	3
E, Encoder Magnético AS5600[4].....	3
F, RTC DS3231[5].....	3
G, LEDS.....	4
H, Motor.....	4
VI. SOFTWARE.....	4
A, Tareas.....	4
B, Control PID.....	4
C, Datalogger.....	5
VII. PCB.....	5
A, Esquemático.....	5
B, Diseño PCB.....	5
C, Test Points.....	6
VIII. CONCLUSIONES.....	6
IX. REFERENCIAS.....	6
Anexo 1 - Diagrama de Tareas.....	7
Anexo 2 - Esquemático.....	8
Anexo 3 - PCB 3D.....	9
Anexo 4 - Manual de usuario.....	10
Anexo 5 - Imagen del Proyecto.....	13

A. Problemática

En varias aplicaciones industriales o de automatización es necesario el control de la posición de un motor, esto con el fin de poder mantener una posición constante frente a perturbaciones del sistema y/o variaciones en la carga, lo cual es clave para aplicaciones donde la posición del motor es crítica.

B. Fundamentación Teórica de Hardware

Para lograr un control de posición preciso, es necesario conocer en tiempo real la posición del eje del motor. Para ello, se empleó un encoder magnético el cual convierte el movimiento mecánico en señales eléctricas, utilizando campos magnéticos para detectar la posición y el movimiento de un eje giratorio o lineal.

El actuador seleccionado es el doble puente H L298N[1], que permite controlar tanto el sentido de giro como la velocidad del motor. La dirección de giro se determina activando los transistores internos del puente H, y la velocidad se regulará mediante una señal PWM que ajusta la potencia entregada al motor.

La información del sistema se visualiza en una pantalla LCD 16x2, además, se cuenta con un teclado 4x4 para la interacción con el usuario lo cual permite ajustar los parámetros del proyecto.

Para la implementación de datalogger, se utiliza un RTC, que proporciona la referencia temporal al momento del registro de los datos. Los datos se almacenan en una memoria EEPROM, que es parte del módulo del RTC, en la cual se guardan tanto resultados como configuraciones.

C. Fundamentación Teórica de Software

El sistema se ha desarrollado sobre un sistema operativo en tiempo real llamado FreeRTOS[2], el cual permite la utilización de tareas concurrentes que se sincronizan mediante mecanismos de sincronización como colas y/o semáforos.

Para el manejo de los recursos de I2C, el cual es utilizado

tanto por el encoder magnético, como por el LCD 16x2 y el RTC/EEPROM, se trabajó con un semáforo que administra el acceso al recurso, con el fin de evitar que 2 dispositivos quieran utilizar los recursos de manera simultánea.

III. ALCANCE

- Medir la posición del motor con una tolerancia de $\pm 5^\circ$ y una actualización cada 20ms.
- Implementar un control PID que permita al motor alcanzar la posición establecida como set point.
- La señal de entrada puede ser trabajada tanto como un escalón como con una rampa.
- Indicar visualmente si se encuentra dentro del error aceptado mediante LED's
- Configurar, mediante el teclado 4x4 las variables del sistemas: Setpoint y Pendiente (si es necesaria)
- Accionar mediante el teclado el prendido y apagado del control PID.
- Almacenar en una memoria EEPROM las distintas configuraciones ingresadas y resultados obtenidos.

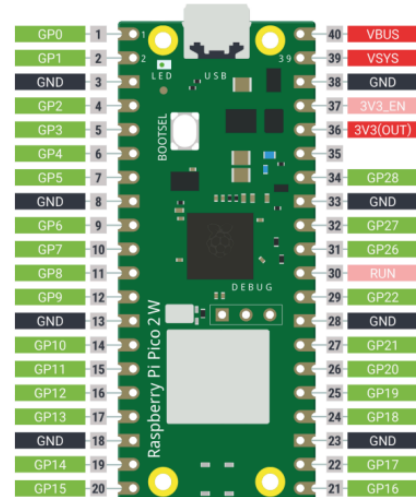


Fig. 2. Microcontrolador Raspberry Pi Pico 2 W Pinout

B, Puente H L298N

Cuenta con dos puentes H integrados, de los cuales utilizaremos solo para el control del motor.

- Voltaje de alimentación del puente H (VS): 5V a 35V
- Corriente máxima por canal: 2A
- Voltaje lógico (VSS): 5V a 7V
- Control de las entradas mediante GPIO's

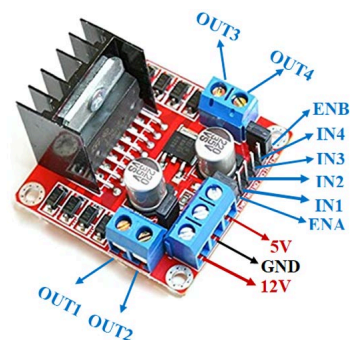


Fig. 3. Puente H L298N Pinout

IV. DIAGRAMA EN BLOQUES

Debajo se pueden observar los módulos que componen al sistema, sus respectivos pines y conexión.

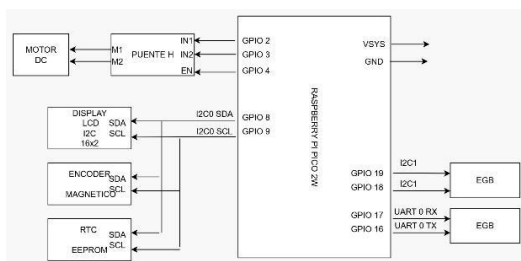


Fig. 1. Diagrama en Bloques.

V. HARDWARE

A, Stick Raspberry Pi Pico 2W

Sus características son:

- Microcontrolador RP2350[3] (Cortex-M33).
- Frecuencia de funcionamiento de hasta 150MHz.
- Tensión de alimentación de 5V, regulada internamente a 3.3V para el RP2350.
- 26 GPIO disponibles.
- 2 controladores SPI.
- 2 controladores I2C.
- 24 canales PWM.

C. Display LCD 16x2 I2C

Se utiliza en conjunto al adaptador I2C PCF8574. Es la interfaz gráfica del proyecto

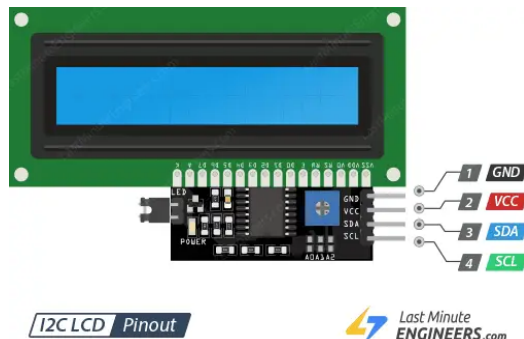


Fig. 4. Display LCD 16x2 I2C Pinout



Fig. 6. Encoder AS5600 Pinout

D, Teclado Matricial 4x4

Se comunica por medio de los GPIO al microcontrolador, se activan con lógica negativa. Es utilizado por el usuario para acceder a la configuración de uso

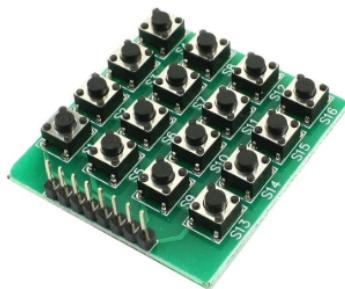


Fig. 5. Teclado Matricial

F, RTC DS3231[5]

Incluye la memoria EEPROM que utilizamos para completar el datalogger.

- Voltaje de funcionamiento: 3.3 – 5.5 VDC
- Precisión del reloj dentro del rango: 0-40°
- Precisión 2 ppm, aproximadamente 1 minuto de error por año
- Comunicación mediante protocolo I2C

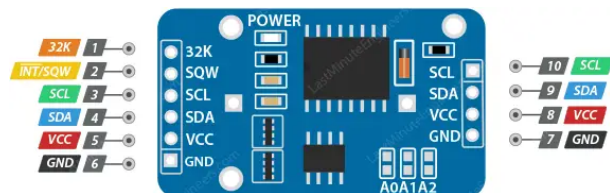


Fig. 7. RTC DS3231 Pinout

E, Encoder Magnético AS5600[4]

- Voltaje de operación: 3.3V.
- Resolución: 12 Bits.
- Posiciones por vuelta: 4096, equivalente a 360 grados.
- Comunicación por protocolo I2C

G, LEDS

Se incluyen en el proyecto 2 LEDs para señalar la banda de error.

H, Motor

Se utiliza un motor arduino de 3.3V-6V con caja reductora, el cual se encastra en un soporte 3D, el mismo permite la interconexión con el encoder AS5600 con su imán generador de campo magnético

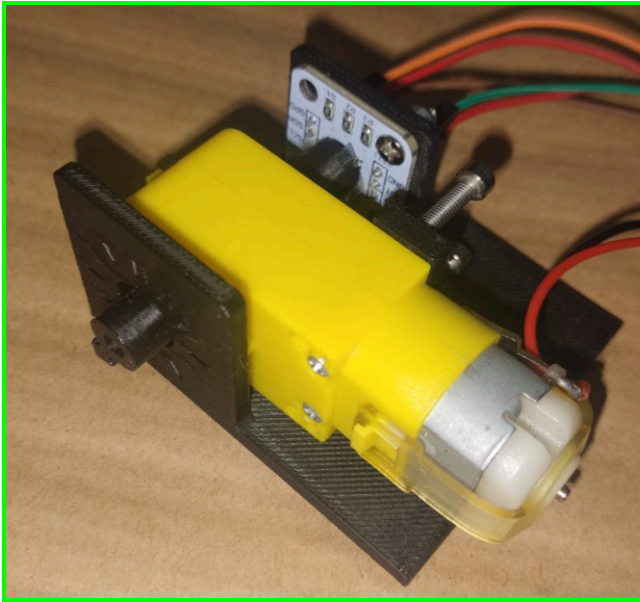


Fig. 8. Motor con Soporte 3D.

VI. SOFTWARE

A, Tareas

El proyecto cuenta con 8 tareas, con una tarea de inicialización que se elimina al ejecutarse. La interacción entre ellas se detalla en el Anexo 1 - Diagrama de Tareas.

La tarea de inicialización inicia tanto los pines, como los módulos, las resistencias de pull up necesarias y crea las colas con las que se trabaja, cuando se completa se elimina y da paso al flujo normal del programa.

Se cuenta con una tarea `task_keyboard` que también hace la función de menú, y sincroniza los pedidos a las tareas de encoder, datalogger, lcd y a la tarea guardiana de i2c, completando la configuración del tipo de señal mediante el input del usuario con el teclado (Esto se detalla en el Anexo 4 - Manual de Usuario). Además, dependiendo de la interacción con el teclado, se comunica con la tarea de control PID para activar o desactivar al mismo.

La tarea guardiana de i2c administra el acceso al recurso del i2c a las tareas del LCD, el RTC, el encoder y el datalogger. Estas tareas se encargan de acuerdo a su orden de: imprimir en pantalla los mensajes, leer la hora actual, efectuar la lectura del ángulo y cargarlo a la cola de ángulos y manejar las acciones referidas al datalogger, ya sea para resultados o configuraciones. La tarea control PID es la que se encarga del control de la salida del motor, asistida por el encoder con el ángulo medido para realizar las correcciones.

También se cuenta con una tarea de LEDs la cual se encarga de definir cuál LED se prende al comparar el ángulo medido con el ángulo seteado.

B, Control PID

La tarea `task_control_pid` implementa una tarea periódica que ejecuta un controlador PID para regular el ángulo del motor. Su objetivo principal es calcular y aplicar una señal de control que lleve al motor a alcanzar un valor de referencia o setpoint definido por el usuario.

Al inicio, se configuran los parámetros del controlador: las ganancias proporcional, integral y derivativa, así como el periodo de muestreo. En nuestro caso, esos valores, luego de probar varias combinaciones hasta lograr una que cumpla satisfactoriamente lo esperado, fueron;

- $K_p=18.0f$
- $K_i=0.2f$
- $K_d=0.1f$
- $T_s=0.01$
- $vel_max = 5.0f$ (Movimiento rápido para pendiente)
- $vel_min = 0.18f$; (Movimiento lento para pendiente)

Luego, dentro de un bucle infinito, la tarea evalúa si el control PID está habilitado mediante una cola (`q_pid_enable`). Si no está activo, apaga el motor y espera hasta la próxima iteración.

Cuando el PID está habilitado, la tarea lee el ángulo actual del motor desde una cola (`q_angulo`) y actualiza la referencia según el tipo de entrada configurado. A partir del ángulo y la referencia, se calculará el error y se ajusta al rango de $[-180^\circ, +180^\circ]$ para evitar problemas de discontinuidad angular.

El controlador luego calcula la componente integral acumulando el error en el tiempo, aplicando una limitación para evitar la saturación del sistema (anti-windup), y calcula la derivada del error. Con estos valores aplica la fórmula

$$Salida = K_p * error + K_i * integral + K_d * derivada$$

La salida está modificando una señal PWM de máximo 2 KHz, la cual separa en 5000 slices positivos y negativos, con el 5000 representando un duty cycle de 100% y el signo representando el sentido del movimiento del motor.

C, Datalogger

La tarea `task_datalogger` actúa manejando las operaciones de lectura y escritura en la memoria EEPROM, según los comandos que reciba a través de la cola `q_datalogger`.

Se llama cuando es necesario guardar o leer datos guardados. Estas solicitudes vienen encapsuladas en un mensaje (`datalogger_msg_t`) que especifica qué tipo de operación debe realizarse (guardar o leer datos de configuración o de

resultados), junto con los datos involucrados y un semáforo opcional para sincronización.

Una vez recibido el comando, la tarea prepara una estructura `i2c_guard_t` y la envía a la cola `q_i2c_guard`, que se encarga de ejecutar físicamente la operación sobre el dispositivo correspondiente. Luego, espera que la operación finalice usando un semáforo, y si el remitente del mensaje había pasado un semáforo para ser notificado al finalizar, la tarea lo libera.

VII. PCB

A, Esquemático

Se encuentra en el Anexo 2 - Esquemático. Fue desarrollado en Kicad 9.0

B, Diseño PCB

Se detalla en el Anexo 3 - Diseño PCB, a la hora de armar el diseño, de acuerdo al esquemático del L298N, se había planteado una alimentación de 12V con un regulador LM7805 para tener 5V para alimentar el resto de los componentes, finalmente se decidió solo utilizar una alimentación de 5V, por lo cual esa parte del diseño quedó obsoleta.



Fig. 9. Diseño PCB.

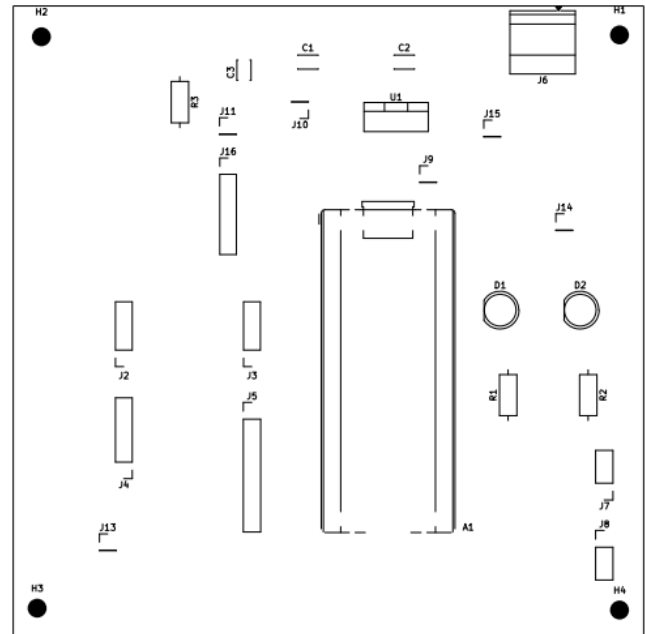


Fig. 10. Vista de Componentes.

C, Test Points

Se plantearon los siguientes Test Points:

- En ENA, para poder observar el PWM que se le aplica al motor desde el control pid.
- En ENA luego de un filtro pasa bajos, para poder observar el PWM como una señal analogica, y poder ver la diferencia entre rampa y escalón.
- En 5V, y 12V (Que tiene 5V actualmente) para poder tener una referencia de la alimentación y poder confirmar que es correcta.

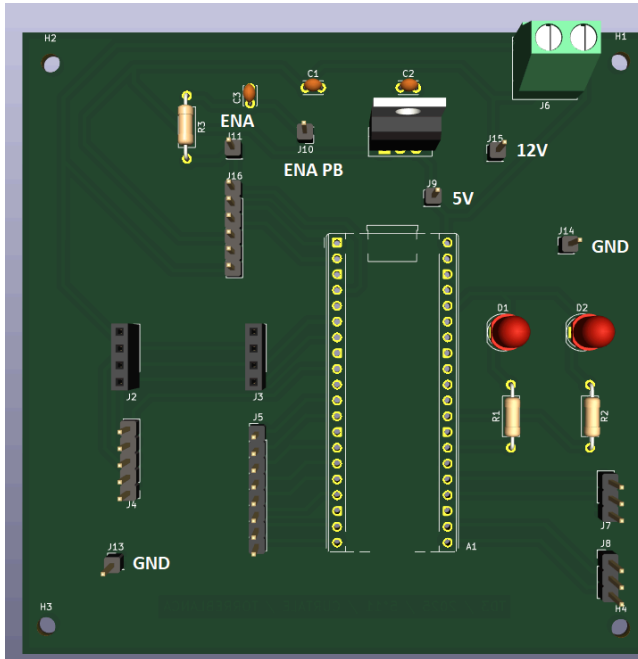


Fig. 10. Vista 3D con Testpoints.

VIII. CONCLUSIONES

Inicialmente, se había propuesto una tolerancia de 3° para el error, luego de comenzar con las mediciones reales, tuvimos que extender esa banda de error a 5° , ya que el PWM generado no alcanzaba a mover al motor cuando se acerca a valores alrededor de los 10° de error, así que tuvimos que bajar la sensibilidad del error. Este error se le puede atribuir al tipo de motor, como cuenta con una caja reductora, no podíamos llegar a hacer el ajuste fino porque el mismo no llegaba a moverse con valores bajos de PWM (Alrededor del 1% al 3% de Duty Cycle).

Con respecto a la respuesta al escalón y la rampa, al escalón responde de buena manera con los valores propuestos, pero a veces con la rampa, cuando planteamos pendientes bajas (Menores al 10%), no llega a generar el PWM para llegar a los valores necesarios algo que podríamos solucionar modificando el valor de K_p , pero al probar distintos valores comenzaba a empeorar la respuesta del escalón, por lo cual decidimos la configuración actual. Una de las mejores que se podría hacer sería empezar a probar distintos sets de valores para conseguir una respuesta más exacta en ambos tipos de señal.

Además, yendo a la parte de diseño, tuvimos el error a la hora de incluir el regulador de línea, ya que finalmente no se terminó utilizando, pero al ya haber hecha la placa decidimos sacarlo de la placa en lugar de rediseñar completamente, ya que no era un elemento crítico.

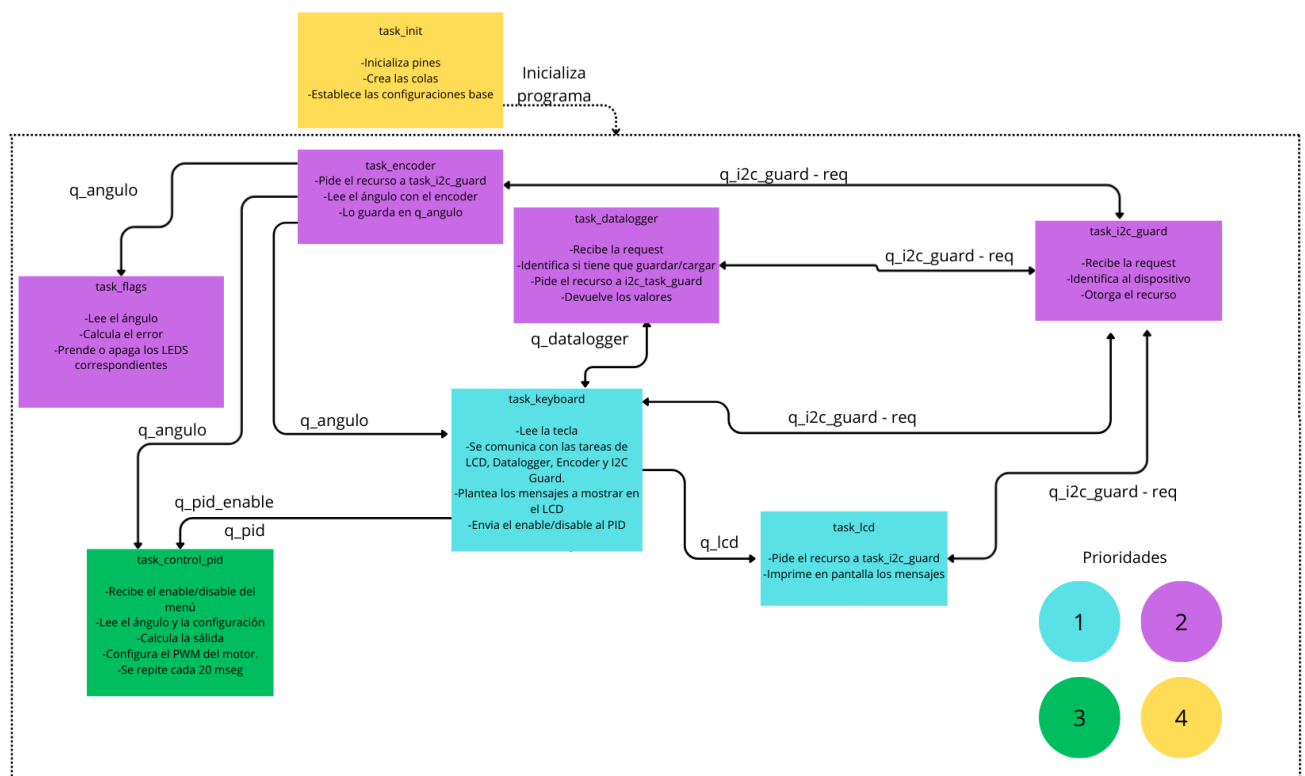
En conclusión, pese a las dificultades y problemáticas

que se presentaron a la hora de realizar el proyecto, finalmente pudimos desarrollar un control de posición de motor de DC con un funcionamiento aceptable, el cual tiene el control PID y datalogger pedidos por el profesor, y aprovecha las características de FreeRTOS

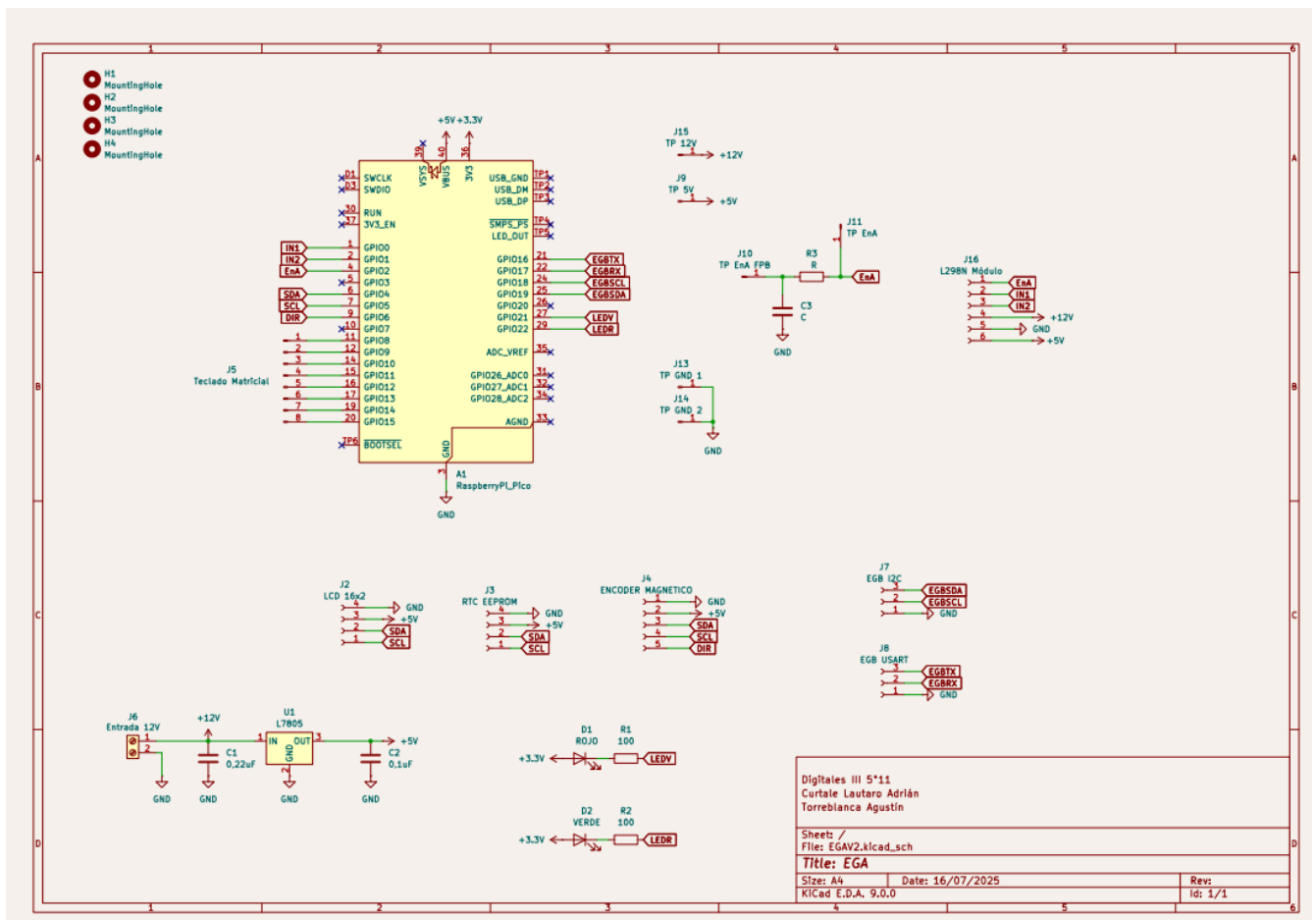
IX. REFERENCIAS

- [1] L298N:
<https://www.alldatasheet.com/datasheet-pdf/pdf/22440/STMICROELECTRONICS/L298N.html>
- [2] FreeRTOS:
<https://www.freertos.org/>
- [3] Raspberry Pi Pico 2W:
<https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>
- [4] AS5600:
<https://www.alldatasheet.com/datasheet-pdf/pdf/621657/AMSC/O/AS5600.html>
- [5] DS3231:
<https://www.alldatasheet.com/datasheet-pdf/pdf/112132/DALLAS/DS3231.html>

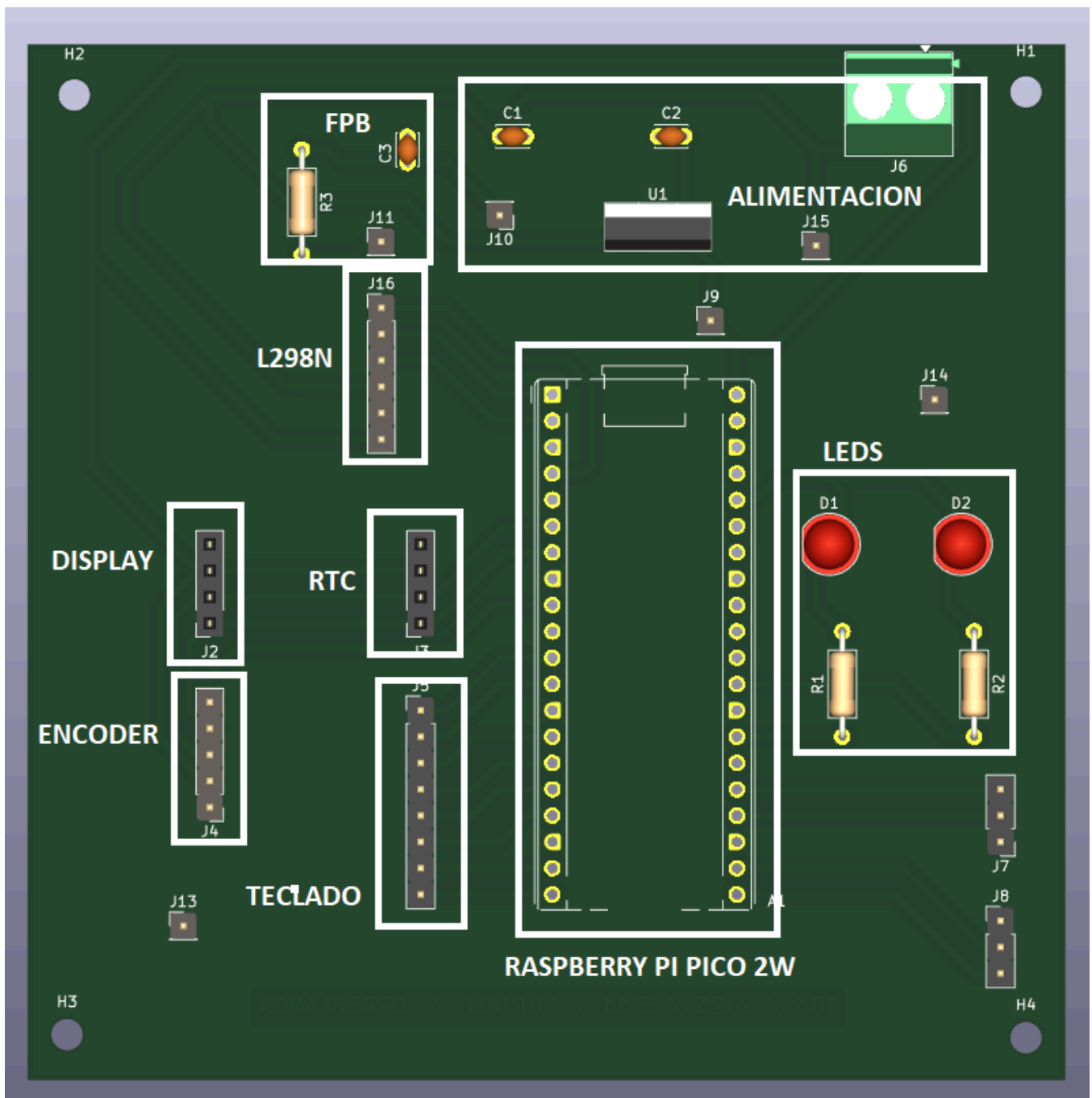
Anexo 1 - Diagrama de Tareas



Anexo 2 - Esquemático



Anexo 3 - PCB 3D



Anexo 4 - Manual de usuario

A continuación se relata la guía de uso para el menú

- ☐ Pantalla de Inicio -> Mensaje Bienvenida (Primer arranque)

En pantalla de inicio:

- Si se presiona 'A': Pantalla de configuración para el tipo de señal, setpoint, y pendiente (en el caso de rampa)



- Si se presiona 'B': Pantalla de monitoreo: ángulo actual, setpoint, error actual y tipo de señal.



- Si se presiona 'C': Pantalla para ver configuración actual: tipo de salida, setpoint y pendiente.



- Si se presiona 'D': Pantalla de muestra de resultados guardados en el datalogger: setpoint, salida PWM Duty Cycle (como OUT, en slices, de -5000 a 5000, representa de -100% a 100%), ángulo medido y fecha (hora y minuto).



- Si se presiona '#': Se activa control PID.



- Si se presiona '*': Se desactiva control PID, guarda resultados actuales en el datalogger.



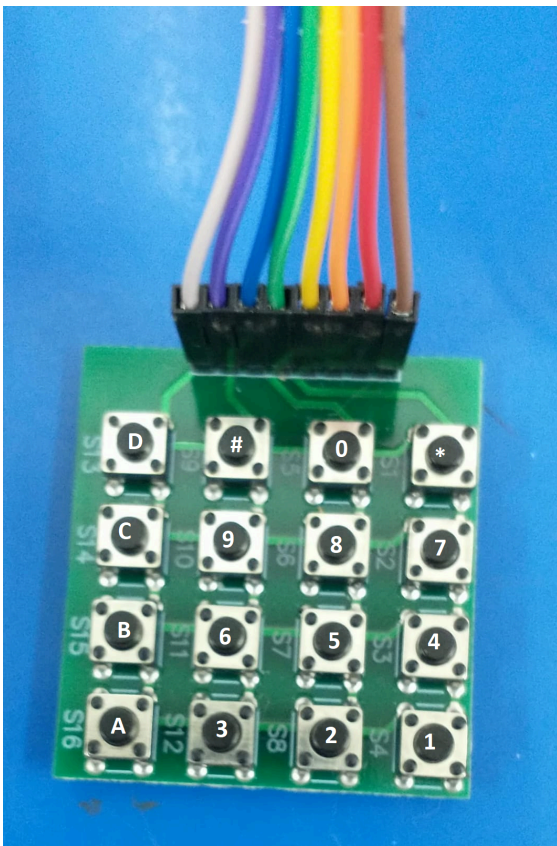
En pantalla de configuración:

- Si se presiona '1': Se elige escalón como tipo de señal.
 - Se pide el ingreso de un setpoint, limitado de 0° a 360° (Rango de medición del AS5600)
 - En caso de excederse en los valores, se limita al valor máximo 360°.
- Si se presiona '2': Se elige rampa como tipo de señal.
 - Se pide el ingreso de un setpoint, limitado de 0° a 360° (Rango de medición del AS5600)
 - Se pide el ingreso de una pendiente, limitado de 1 a 100, representa el porcentaje de velocidad de subida de la rampa, 1 es muy lento, 100 su comportamiento es similar a un escalón.
 - En caso de excederse en los valores, se limitan a los máximos 360° y 100.
- Si se presiona '#': Se guardan los datos / Avanza la pantalla (Solo si es rampa).
- Si se presiona '*': Se cancela la configuración.

En pantalla de monitoreo:

- Si se presiona cualquier tecla: Abandona el modo de monitoreo y vuelve al modo de pantalla de inicio, hay que volver a presionar la tecla del siguiente menú para acceder al mismo.
- ☐ Si se prende el LED Verde: El error actual es menor a 5°
- ☐ Si se prende el LED Rojo: El error actual es mayor a 5°

Guia Teclado:



Anexo 5 - Imagen del Proyecto

